

Lista zadań nr 8

Uwaga: Poniższe zadania (także zadanie 7.) można rozwiązać modyfikując kod z wykładu nr 9 (czyli tego BEZ wyrażeń, których ewaluacja może zmieniać stan programu, o których mówiliśmy na wykładzie nr 10).

Zadanie 1. (2 pkt)

Dodaj do języka WHILE pętlę for w stylu C, czyli konstrukcję postaci

```
for (init; cond; step) body
```

gdzie init i step to instrukcje wykonywane, odpowiednio, jednokrotnie przed pętlą oraz po każdej iteracji jej ciała, a cond to wyrażenie sprawdzane przed każdą iteracją i decydujące, czy pętla powinna się jeszcze raz wykonać. Przykładowo, poniższy kod powinien wypisać liczby od 1 do 10:

```
for (i = 1; i <= 10; i = i + 1) print(i);
```

Postaraj się dodać tę konstrukcję jako lukier syntaktyczny w parserze, tj. bez wprowadzania nowego konstruktora w typie stmt i bez modyfikowania ewaluatora.

Pozwól też na pominięcie dowolnego z trzech „pól” pętli. Pomijając warunek otrzymujemy pętlę nieskończoną, czyli for (;;) body powinno być równoważne while (true) body. Pomijając init albo step pozbywamy się odpowiedniej instrukcji, zachowując pozostałe pola bez zmian.

Zadanie 2. (2 pkt)

W pliku spigot.while znajdującym się na skosie znajduje się następujący program:

```
q = 1;
r = 0;
t = 1;
k = 1;
n = 3;
l = 3;

while (true) {
    if (4 * q + r - t < n * t) {
```

```

    print(n);
    print("\n");
    qq = 10 * q;
    rr = 10 * (r - n * t);
    tt = t;
    kk = k;
    nn = ((10 * (3 * q + r)) / t) - (10 * n);
    ll = 1;
} else {
    qq = q * k;
    rr = (2 * q + r) * 1;
    tt = t * 1;
    kk = k + 1;
    nn = (q * (7 * k + 2) + r * 1) / (t * 1);
    ll = 1 + 2;
}
q = qq;
r = rr;
t = tt;
k = kk;
n = nn;
l = ll;
}

```

Program ten wypisuje nieskończony ciąg bezsensownych liczb. Są jednak one bezsensowne tylko dlatego, że zmienne występujące w programie szybko zaczynają się przepęlniać – maszynowe inty są dla nich po prostu za małe. Użyjmy więc większych.

Zmodyfikuj interpreter języka WHILE tak, by zamiast ze standardowych intów, korzystał on z liczb typu `Bigint`.¹ t udostępnianych przez bibliotekę `bignum`. Czy teraz rozpoznajesz ciąg liczb wypisywany na ekran przez powyższy program?

Uwaga: Bibliotekę `bignum` można zainstalować poleceniem

```
opam install bignum
```

By nasz program widział bibliotekę, należy dołączyć ją do listy zależności projektu, modyfikując plik `src/dune` tak, by wyglądał następująco:

```

(library
  (name while)
  (libraries bignum))

(menhir (modules parser))

(ocamllex lexer)

```

¹<https://ocaml.org/p/bignum/latest/doc/Bigint/index.html>

Wskazówka: Można zmniejszyć liczbę modyfikacji w kodzie używając składni w OCamlu

```
Moj_modul.(wyrażenie)
```

która sprawia, że lokalnie otwieramy moduł `Moj_modul` w wyrażeniu `wyrażenie`.

Zadanie 3. (2 pkt)

Tablica (nie mylić z listą!) to struktura danych przechowująca elementy indeksowane liczbami całkowitymi. Dodaj do języka WHILE (parser, lekser, AST, wartości, ewaluator) typ tablic takich że:

- Tablica to wartość i może przechowywać dowolne wartości,
- Tablica jest nieskończona, ale tylko na skończenie wielu indeksach wartości są różne od zera,
- Tablicę można stworzyć przy pomocy składni konkretnej $\{e_0; e_1; \dots; e_{n-1}\}$. Wartością tego wyrażenia jest tablica, która na indeksie 0 ma wartość wyrażenia e_0 , na indeksie 1 ma wartość wyrażenia e_1 itd. aż do wyrażenia e_{n-1} , którego wartość znajduje się na indeksie $n-1$. Wartości na pozostałych indeksach tej tablicy są zerami (czyli wartościami `VInt 0`).
- W składni abstrakcyjnej powyższy konstruktor tablicy reprezentuj jako listę wyrażeń:

```
type expr =  
  ...  
  | Array of expr list
```
- Wartość reprezentującą tablicę zaimplementuj przy użyciu słownika `Map.Make(Int).t`, czyli podobnego do tego, którego używaliśmy dla pamięci i środowisk, ale którego kluczami są inty.

Nie zapomnij odpowiednio rozszerzyć funkcji `show_value`.

Wskazówka: Przejrzyj moduł `Map.Make` w poszukiwaniu funkcji, które mogą się przydać. Autorowi zadania przydały się `of_list` i `max_binding`.

Zadanie 4. (2 pkt)

Do języka WHILE z poprzedniego zadania dodaj instrukcję, która pozwala ustawić wartość przechowywaną pod danym indeksem w tablicy:

```
type stmt =  
  ...  
  | AssignArray of ident * expr * expr
```

a także wyrażenie pozwalające na odczytanie wartości z tablicy na danym indeksie:

```
type expr =  
  ...  
  | ArrayRead of ident * expr
```

Składnia konkretna może być standardowa, czyli $xs[i] = e$ dla AssignArray i $xs[i]$ dla ReadArray. Jeśli wartość zmiennej xs nie jest tablicą, próba wykonania instrukcji $xs[i] = e$ albo obliczenia wartości wyrażenia $xs[i]$ powinna zakończyć się błędem kończącym działanie interpretera.

Uwaga: W wersji trudniejszej, możesz zrobić odpowiednio zagnieżdżone wersje tych instrukcji i operacji, np. $xs[k][n]$, czyli odczytanie wartości spod n -tego indeksu z tablicy, która przechowywana jest pod k -tym indeksem w tablicy xs .

Zadanie 5. (1 pkt)

Zaimplementuj w języku WHILE dowolny algorytm sortujący tablicę daną jako zmienna xs , której towarzyszy zmienna $length$ mówiąca, jak długi prefiks tablicy należy posortować. Przykładowo, program

```
xs = {7;2;1;6;5;3;4};  
length = 7;
```

(tu wpisz kod...)

```
print(xs);  
print("\n");
```

powinien wypisać na ekran wartość {1;2;3;4;5;6;7}.

Zadanie 6. (1 pkt)

Zaimplementuj w języku WHILE ewaluator wyrażeń w odwrotnej notacji polskiej² reprezentowanych w formie tablicy zawierającej liczby albo stringi reprezentujące operacje (dla uproszczenia założmy, że jedyne operacje, jakie mamy, to $*$ i $+$) oraz wartość mówiąca, jak długi prefiks tablicy zajmuje wyrażenie. Przykładowo, poniższy program powinien wyświetlić na ekranie liczbę 16.

²https://pl.wikipedia.org/wiki/Odwrotna_notacja_polska

```
expr = { 1; 9; "+"; 2; 3; "*"; "+" };  
length = 7;
```

```
stack = { };
```

```
(tu wpisz kod...)
```

```
print(stack[0]);  
print("\n");
```

Założ, że wejściowa tablica zawsze zawiera poprawne wyrażenie.

Wskazówka: Wewnątrz WHILE'a nie umiemy sprawdzić, czy coś jest intem czy stringiem, ale możemy porównać wartości do konkretnych stałych. Jeśli element tablicy expr nie jest wartością "+" ani "*", to możemy śmiało założyć, że jest intem.

Zadanie 7. (2 pkt)

Podobnie jak state z wykładu, wiele innych konstruktorów typów danych ma swoje funkcje pure i bind. Przykładowo, dla konstruktora typu option są one zdefiniowane jako:

```
type 'a option =  
  | None  
  | Some of 'a  
  
let pure (a : 'a) : 'a option =  
  Some a  
  
let bind (o : 'a option) (f : 'a -> 'b option) : 'b option =  
  match o with  
  | None -> None  
  | Some v -> f v
```

Wy tłumacz, co robią te funkcje i użyj ich do rozwiązania zadania 7. z listy 7. Spróbuj użyć let*.